

HOWARD UNIVERSITY • DEPARTMENT OF MATHEMATICS

Homework 1: Data Wrangling with Pandas

MATH 014 • Introduction to Data Science • Fall 2026

Instructor: Dr. Moussa Doumbia • mdoumbia@howard.edu

Dataset:	Total: 100 points	Due: _____
<code>week01_customers.csv</code>		
Submit on: Canvas	Format: .ipynb + PDF	Weight: Homework (20%)

1. Overview

You have been handed a customer file exported from a company's billing system. It contains 57 rows and 8 columns, and essentially every column has something wrong with it. Your job is to turn it into a dataset you would be willing to attach your name to.

This assignment is not about memorizing pandas methods. It is about the habit underneath them: **look before you transform, and write down why you did what you did**. A cleaning decision without a stated reason is not a finished answer.

Learning outcomes. On completing this assignment you should be able to:

- (a) Profile an unfamiliar dataset and produce evidence of its defects before altering it.
- (b) Convert text columns to correct types without silently discarding recoverable values.
- (c) Distinguish genuine missingness from sentinel values, disguised nulls, and impossible measurements.
- (d) Package a cleaning process as a reproducible, self-validating function.
- (e) Document what you fixed, what you decided, and what you could not resolve.

The dataset

Column	Should contain	Notes
<code>customer_id</code>	Identifier	Format C-1001
<code>name</code>	Full name	First and last
<code>age</code>	Integer	Customer age in years
<code>city</code>	Category	One of five US cities
<code>signup_date</code>	Date	When the customer joined
<code>price</code>	Currency	Purchase amount in USD
<code>email</code>	Text	Contact address
<code>membership</code>	Category	bronze, silver, or gold

Setup

Create a new Jupyter notebook named `lastname_hw01.ipynb` in the same folder as the CSV. Begin with this cell, exactly as written:

```
import re
```

```
import numpy as np
import pandas as pd

raw = pd.read_csv('week01_customers.csv', dtype=str, keep_default_na=False)
print(raw.shape)
```

Do not remove `keep_default_na=False`. By default, pandas silently converts 'N/A', 'n/a' and '' into NaN before you ever see them. This file uses *four* different conventions for missing data, and the whole point of Part I is to discover that yourself. Turning off the automatic conversion is what makes the discovery possible.

2. Part I — Inspect Before You Touch Anything

[15 pts]

AIAS LEVEL 1 • NO AI

Complete this part with no AI assistance of any kind. You are producing the evidence that will justify every decision you make later.

Task 1. Shape and types

[3 pts]

Report the number of rows and columns. Run `raw.dtypes` and state why every column is `object`. Then name the two columns that *should* be numeric and explain, in one sentence each, what is preventing that.

Task 2. Missing values

[4 pts]

Count the missing values in each column and express each as a percentage of rows. Report your results in a small table. Then answer: does `raw.isna().sum()` give you the true picture? Justify your answer by examining the actual distinct values in `price`.

Task 3. Rows versus customers

[4 pts]

Compare `len(raw)` with `raw['customer_id'].nunique()`. Identify every repeated ID. For each, determine whether the repeated rows are identical across all eight columns or differ somewhere, and show the code that proves it.

Task 4. How many cities?

[4 pts]

List every distinct value of `city` with its count. The obvious answer to “how many cities are in this dataset?” is wrong. Give both the naive number and the correct number, and present the evidence that resolves the difference. Explain the consequence: if you grouped by the raw `city` column, how would it change which city looks largest?

3. Part II — Clean One Column at a Time

[30 pts]

AIAS LEVEL 2–3

Attempt each function yourself first. You may then use an approved AI tool to compare approaches or debug, provided you record the prompt and what you changed. Write each cleaner as a *named function*, and immediately below it include a test cell showing your function’s output on at least five inputs, including at least one that should return NaN.

Task 5. A null normalizer

[4 pts]

Write `to_null(value)` that returns `np.nan` for any token meaning “no value,” and otherwise returns

the value stripped of surrounding whitespace. Define your set of null tokens as a named constant at the top of your notebook, not inline. List every token you chose to include and state why you included it.

Task 6. Price [6 pts]

Write `clean_price(value)` converting the `price` strings to floats. It must handle dollar signs, the USD prefix, thousands separators, and stray whitespace. Then answer precisely:

- How many prices are unusable *after* conversion?
- Which specific `customer_id` is unusable but would *not* have been flagged by a missingness check run before conversion?
- State the general lesson this row teaches about the order of profiling and type conversion.

Task 7. Age, in two separate steps [8 pts]

Write *two* functions: `clean_age(value)`, which converts to a float, and `validate_age(series)`, which blanks implausible values.

- Explain why converting a type and judging a value are kept as separate functions.
- Compare `pd.to_numeric(raw['age'], errors='coerce')` against your `clean_age`. Report the NaN count from each and list exactly which values the naive call discarded.
- Choose a plausible age range. State both bounds and defend each with a reason specific to this dataset — not “because it seems right.”
- Report the mean age before and after applying your range, and state how much two rows distorted it.

Task 8. City [5 pts]

Write `clean_city(value)` producing one canonical label per city. Report the final counts. Then explain why `.str.title()` alone is insufficient, naming the specific value it fails on and what it produces instead.

Task 9. Dates [4 pts]

Convert `signup_date` to datetime in a single pass. Identify every row not in ISO YYYY-MM-DD form and name the format each uses. Confirm your parse produced zero failures and report the date range.

Then consider the value 01/23/2026. Explain why it is unambiguous, and construct a different date string that *would* be ambiguous and describe the two readings.

Task 10. Email and name [3 pts]

Write a regular expression that checks basic email structure. Report how many addresses are missing and how many of the present ones pass. Add a boolean column `email_valid` rather than dropping any rows, and justify that choice in one sentence.

Then split `name` into `first_name` and `last_name`, and verify that every row splits into exactly two parts. Give two examples of real names that would break your approach.

4. Part III — Build a Reproducible Pipeline [20 pts]

AIAS LEVEL 2–3

Task 11. The `clean()` function [12 pts]

Package everything from Part II into a single function `clean(raw) -> DataFrame` that runs start to finish on the raw file. It must:

- (a) begin with `raw.copy()`;
- (b) convert all columns to their correct types;
- (c) make `membership` an *ordered* categorical (`bronze < silver < gold`);
- (d) remove duplicate rows;
- (e) return columns in a sensible order.

In a markdown cell, explain why `raw.copy()` on the first line matters. Describe concretely what goes wrong without it when a cell is run twice.

Task 12. Deduplication, examined [4 pts]

Apply both `drop_duplicates()` and `drop_duplicates(subset='customer_id')`. Report both row counts and explain why they agree on *this* file. Then describe a one-sentence scenario in which they would disagree, and state which of `keep='first'` or `keep='last'` would be correct in that scenario.

Task 13. Validation [4 pts]

Write a function `validate(df)` containing at least six `assert` statements that would fail if a future version of this file broke an assumption. At least two must check *dtypes* and at least two must check *value ranges*. Explain in one sentence why an assertion that never fails is still worth writing.

5. Part IV — Analysis on the Cleaned Data [15 pts]

AIAS LEVEL 2–3

Every number in this part must be computed on your cleaned, deduplicated DataFrame. Report the denominator alongside every statistic.

Task 14. Price by membership tier [6 pts]

Compute count, mean, and median `price` by membership tier. You would expect `gold > silver > bronze`. Test that expectation against *both* statistics and report exactly what you find.

If the two statistics disagree, explain what property of the underlying distribution causes it, and state which statistic you would put in a report to a manager and why.

Task 15. Price by city [5 pts]

Compute count, mean, and median `price` by canonical city. Identify the city with the highest mean and the city with the highest median. Explain the disagreement and state what you would report.

Task 16. Signups over time [4 pts]

Count signups per calendar month. Identify the largest month. Then explain why this series must *not* be reported as evidence of growth, and propose a statistic that would be defensible instead.

6. Part V — Judgment and Documentation [20 pts]

AIAS LEVEL 4

These questions have no single correct answer. They are graded on the quality of your reasoning and the honesty of your documentation. **This part requires the three AI deliverables listed in Section 6.**

Task 17. Are 999 and -3 the same kind of error?

[5 pts]

Both are impossible ages, but they likely have different causes. Propose an explanation for each. State what specific information you would need in order to confirm your explanation, and where you would obtain it. Finally, explain why the distinction changes what you write in your report, even though both values become NaN in your code.

Task 18. What does ‘-’ mean?

[5 pts]

Customer C-1041’s price is the single character ‘-’. Identify at least three plausible meanings. For each, state how you would encode it and what it would do to the mean price. Commit to one interpretation and defend it. Finally, explain why the dangerous version of this decision is the one made *implicitly* by a `float()` call.

Task 19. Errors or legitimate repeat records?

[5 pts]

Argue that the duplicated rows are data-entry errors. Then argue that they are legitimate second purchases. Point to a *specific column* whose presence and content settles the question, and say which reading you find stronger. State what you would write in your report to preserve the ambiguity honestly.

Task 20. The data quality report

[5 pts]

Write a one-page report in a markdown cell, addressed to a colleague who will use your cleaned file but did not clean it themselves. It must contain:

- (a) a table of at least five defects with scope, severity, and the action you took;
- (b) the headline figures a reader needs (final row count, cities, date range, and the denominator for age and price);
- (c) a section titled **Unresolved**, listing at least four questions you could not answer without contacting the data owner.

A report claiming the data is now *clean* will not receive full credit. The data is now **usable, with documented limitations**. Stating what you could not resolve is not an admission of failure — it is the part that lets the next person trust everything else you wrote.

7. Required AI Documentation

Parts I through IV may be completed without AI. **Part V is an AI-permitted (Level 4) task**, and per the course syllabus your submission must include all three of the following. Submissions missing any one of them will not receive credit for Part V.

1. **Pre-AI artifact.** Your own written answer to *each* Part V question, produced *before* opening any AI tool. Timestamp it in a markdown cell.
2. **AI process log.** For each interaction: the exact prompt, a summary of the response, and your decision to **accept**, **modify**, or **reject** it — with a one-sentence reason.
3. **Metacognitive reflection** (200–300 words). What did AI improve? What did it get wrong or miss? How did you verify its output against the actual data? Identify one suggestion you accepted, one you modified, and one you rejected.

You remain fully responsible for the correctness of every number, method, and explanation you submit. AI output can be incomplete, misleading, or simply wrong. Uploading this dataset to an AI tool is permitted because it contains no real personal data; **never** upload private, FERPA-protected, or personally identifiable data to any AI tool.

8. Submission Checklist

- ☐ Notebook named `lastname_hw01.ipynb`, run top to bottom with **Kernel** → **Restart & Run All** before saving, with all outputs visible.
- ☐ A PDF export of the executed notebook.
- ☐ `customers_clean.csv`, your cleaned output file.
- ☐ Every cleaning function defined with a docstring and a test cell below it.
- ☐ Markdown explanations for all written questions — not code comments.
- ☐ Pre-AI artifact, AI process log, and reflection for Part V.
- ☐ No hard-coded answers. Every number in your write-up produced by code in the notebook.

9. Grading Rubric

Component	Pts	What earns full credit
I. Inspection	15	Defects found <i>and</i> evidenced with code output, not asserted.
II. Column cleaning	30	Named functions with test cells; recoverable values preserved; disguised nulls caught.
III. Pipeline	20	Runs end to end from raw; non-mutating; assertions that would actually fail.
IV. Analysis	15	Correct figures with denominators stated; mean/median divergence identified and explained.
V. Judgment	20	Positions defended with evidence from the data; unresolved items named honestly.
Total	100	

How this is really graded. Two students can produce identical cleaned files and receive very different scores. The one who writes “dropped 5 rows with missing price” earns less than the one who writes “left 5 missing prices as NaN rather than imputing; with no evidence about why they are missing, imputation would invent transactions that never occurred — mean reported on 50 of 55 customers.” **Judging your own choices is the skill this course builds.**